Managing Memory 1

Allocating and Deallocating Memory

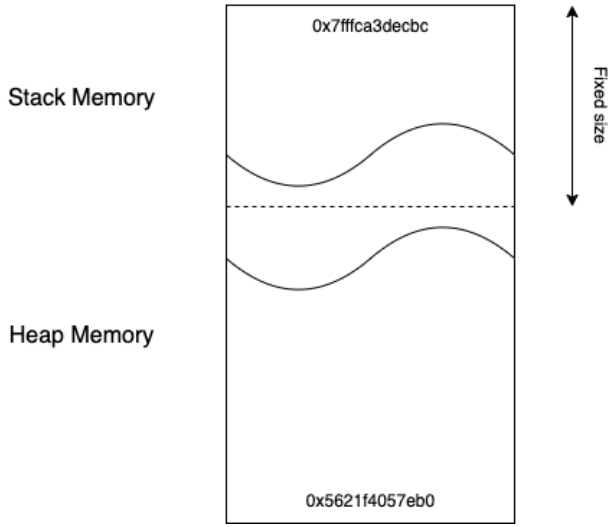
Department of Computer Science
University of Pretoria

7 May 2024

There are two types of memory:

- Stack memory - fixed size, higher in the primary memory address space, allocation and deallocation of memory is automatic
- Heap memory - variable size, lower in the memory address space, programmer allocates and deallocates the memory

As the program allocates stack and heap memory, the stack and heap addresses approach each other in the available memory space.

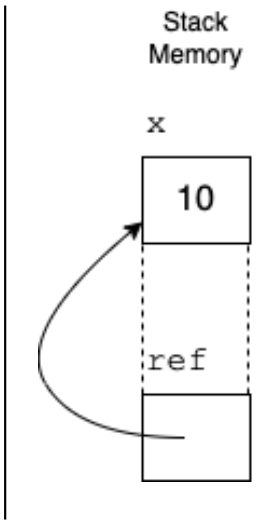


- When stack and heap memory meet, there is no more memory available too allocate.
- If no deallocations have occurred to free memory, the program crashes due to being out of memory.

- Stack memory is a part of main memory that is used to allocate variables in a first-in-last-out fashion.
- The memory is allocated at run-time.
- When a variable goes out of scope the memory is automatically deallocated.

A reference is another name for an existing variable. It links to the memory address where the value represented by the variable resides

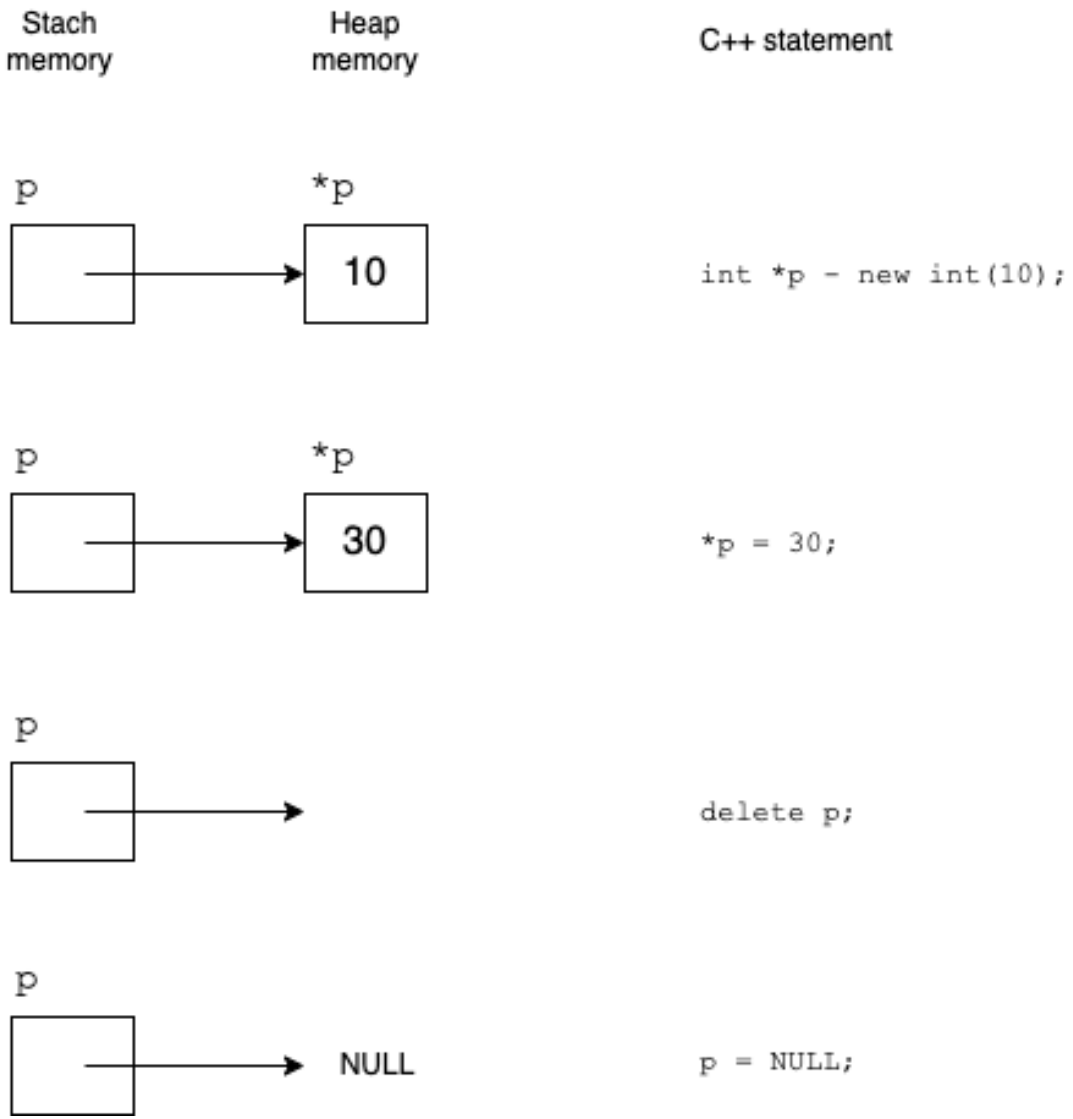
```
int x = 10;
int &ref = x;
```



- Allocation of heap memory is done using the `new` construct and deallocation with the `delete` construct defined in C++.
- Inappropriate management of heap memory can lead to run-time errors such as segmentation faults.

- Heap memory is allocated dynamically.
- The size of the item to be allocated and the lifetime are not known at compile-time, but are determined at run-time.
- It is accessed with a pointer.

- A pointer is a variable that resides in stack memory that holds a memory address pointing to a location in heap memory
- To allocate enough memory on the heap for the specific type of value to be stored, `new` is used
- To access the value store, the pointer variable needs to be dereferenced by applying the operator `*`
- To deallocate the memory on the heap, `delete` is used
- It is good programming practice to "`null`" the variable after deletion to enable comparison against a known value. This can be done by assigning 0, NULL or nullptr to the variable


Stach memory
Heap memory
C++ statement
p
*p
10
int *p - new int(10);
p
*p
30
*p = 30;
p
delete p;
p
NULL
p = NULL;

In Lecture 24 we considered passing parameters by reference. We did not discuss passing parameters as pointers, nor did we discuss functions returning references (and now pointers).

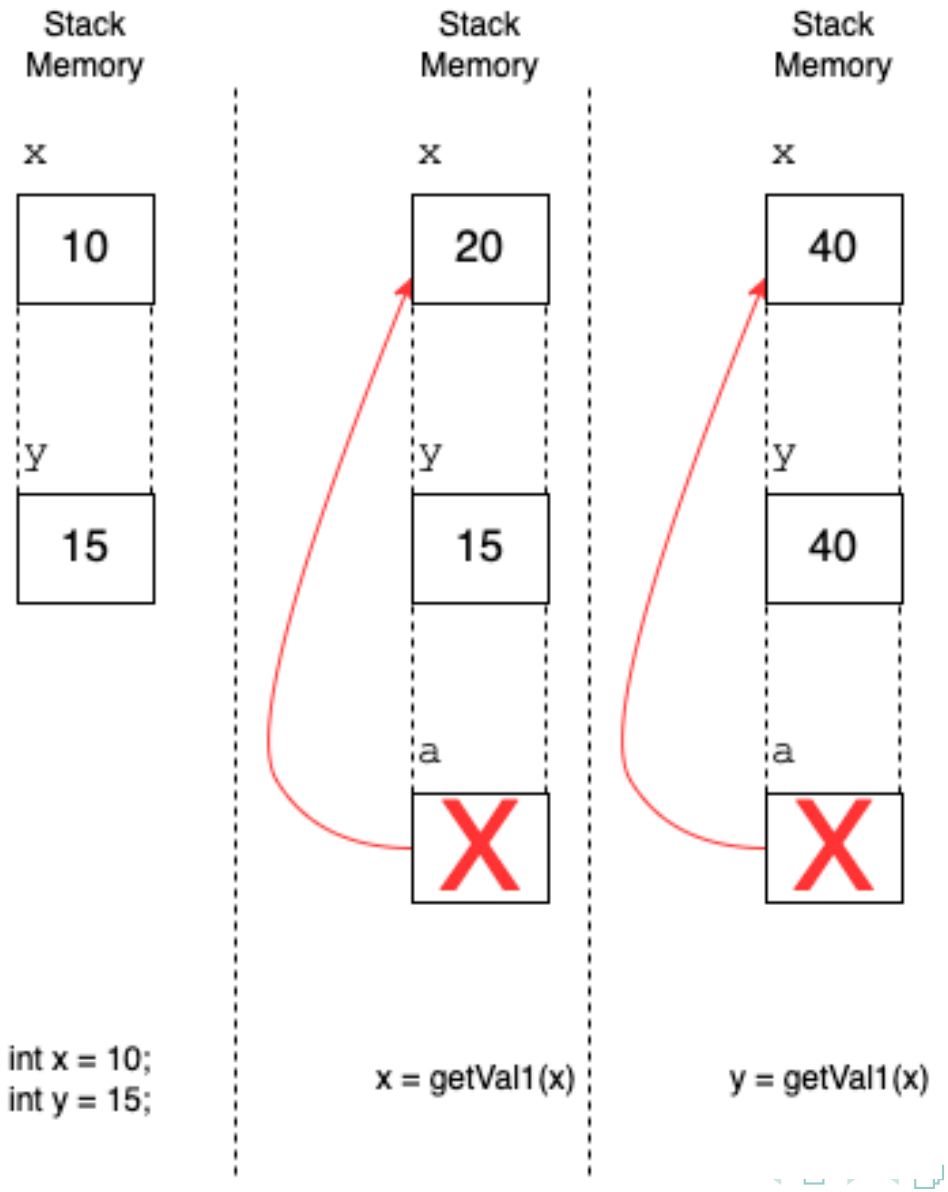
To return a reference to the value, the variable holding the value must have been created before the function is called (refer to `getVal1`). That is, it cannot be defined in the function (refer to `getVal2`). Why?

```
int& getVal1(int& a) { // Valid C++
    a *= 2;
    return a;
}

int& getVal2() { // Invalid C++
    int b = 10;
    return b;
}
```

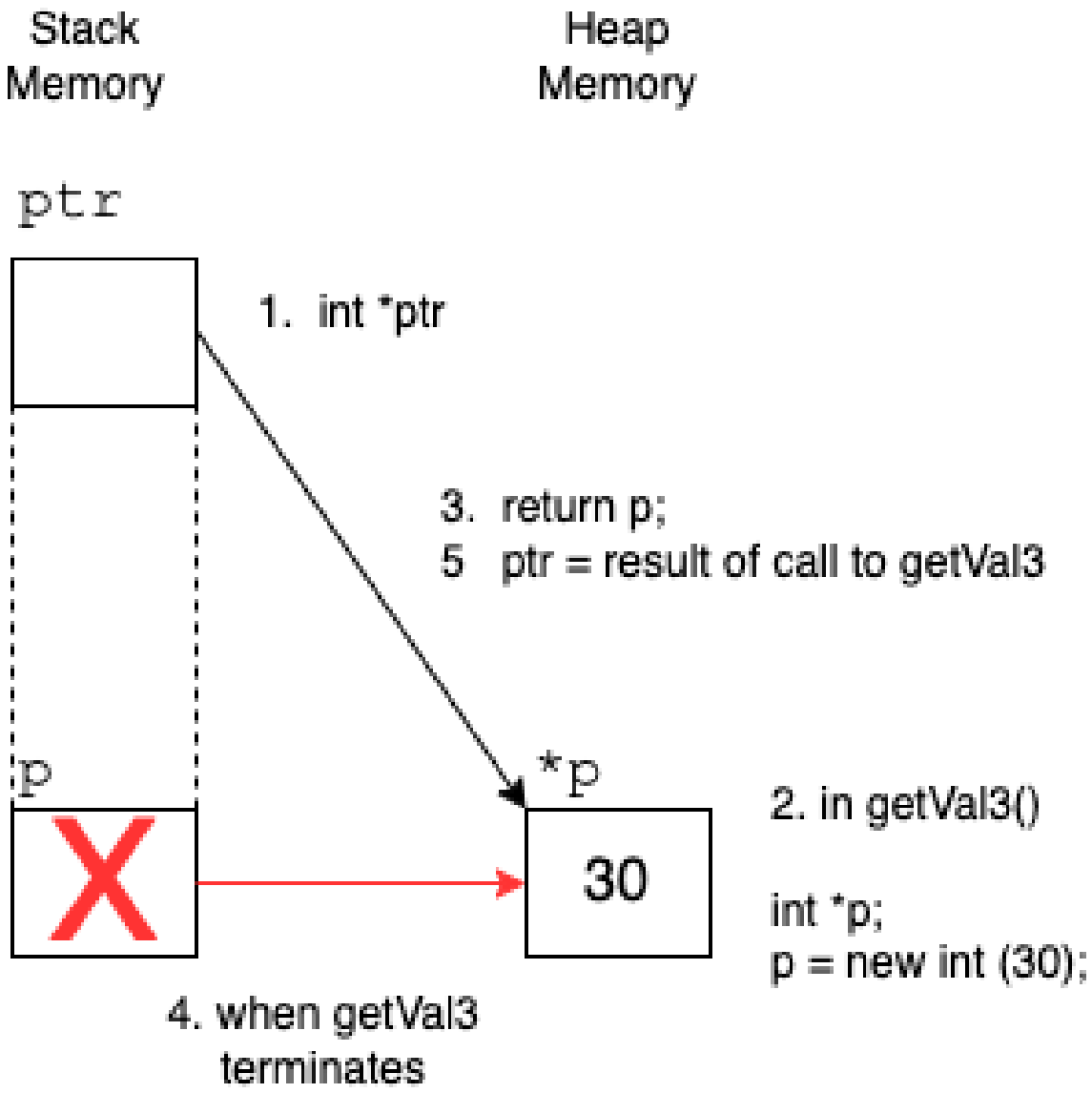
What would the result be if `getVal1` is called in the following two ways:

```
int& getVal1(int& a) { // Valid C++
    a *= 2;
    return a;
}
...
int x = 10;
int y = 20;
x = getVal1(x);
y = getVal1(x);
```


Stack Memory
x
10
y
15
int x = 10;
int y = 15;
Stack Memory
x
20
y
15
a
X
x = getVal1(x)
Stack Memory
x
40
y
40
a
X
y = getVal1(x)

Consider the following code that defines and allocates memory for a value on the heap.

```
int* getVal3() {
    int* p;
    p = new int(30);
    return p;
}
...
int* ptr = getVal3();
...
delete ptr;
ptr = NULL;
```


Stack Memory
Heap Memory
ptr
1. int *ptr
3. return p;
5 ptr = result of call to getVal3
p
*p
30
2. in getVal3()
int *p;
p = new int (30);
4. when getVal3 terminates

When a function allocates memory on the heap and returns the pointer to the memory on the heap, the onus is on the code that has called the function to clean (`delete`) the heap memory and no longer on the function that called `new`.

What about passing a pointer as a parameter? Consider the definition of `getVal4` given as

```
void getVal4(int *p);
```

Questions to ask.

- Where should the memory be allocated that `p` points to?
- Where should the memory be deleted?
- How is this function called if the variable acting as the actual parameter is referring to stack memory? Can there be an allocation of memory in the function then?
- How is this function called if the variable acting as the actual parameter is defined as a pointer pointing to heap memory?

Remember:

- All memory that has been allocated with a `new` by the programmer must be deallocated using `delete`
- Using `delete` will release the memory back to the operating system and free it up for either the same program to reuse or another program to use
- Once the memory has been freed, it is good programming practice to assign "NULL" to the pointer variable